COMP3900/COMP9900

Computer Science Project

P94 – AI Apply: One-Click Job Application Engine

Team Injured (APPLE-H13A)

Lawrence Seeto	z5491885	z5491885@ad.unsw.edu.au	Product Owner
Gobind Singh	z5437859	z5437859@ad.unsw.edu.au	Team Member
Kathy Phan	z5437872	z5437872@ad.unsw.edu.au	Scrum Master
Lachlan Butcher	z5419198	z5419198@ad.unsw.edu.au	Team Member
Aqil Tuan Razali	z5423307	z5423307@ad.unsw.edu.au	Team Member
Samuel Lim	z5376765	z5376765@ad.unsw.edu.au	Team Member

Project Proposal Submission Date: 14/08/2025

Table of Contents

Executive Summary

A basic prototype was our first step: select a job, autofill, and submit. The interface seemed like a dark box, auth was erratic, and users couldn't determine whether it was doing anything. We made the Progress tab the centre of the experience and rebuilt around a straightforward journey: Discover → Prepare → Execute. The finished design includes stability (JWT, state machine, selector registry), predictability (hover indications on Review/Apply), and transparency (live steps). To deliver something dependable, we purposefully limited our reach to two platforms (Seek and LinkedIn Easy Apply) before expanding.

Problem & Design Objectives

Issues noted include ambiguous results, redundant work, brittle authentication, and opaque automation.

Goals:

1. Make the state of the system always apparent.
2. Before submission, give users power.
3. Under MV3, increase stability and maintainability.
4. Scale after launching a reliable core on a small number of platforms.

Evolution of the Design (By Iteration)

Early Prototype

- "Make it work" (detect → fill → submit) is the main objective.
- Problems included duplicating apps, sporadic problems caused by cookie-based authentication, and no status reporting.

Mid-Term (Pass for Stability)

- Switched to JWT (constant, stateless calls).
- The job history included timestamps, session totals, and "Reviewed/Applied" badges.
- The platform selector registry and content-script state machine were included to lessen brittleness.

Conclusion (Control & Clarity)

- The multi-stage Progress tab now functions as both a live stepper (Review/Apply) and a Job History (browsing): Finding → Parsing → Pre-fill → Upload → Verifying → Sending → Result.
- Before clicking, the effects are made evident by hover indications on Review and Apply.
- To stop runaway listeners, use onModalClose() to loop guard.
- Chrome.storage.local stores session data to maintain the accuracy of counters and history.

Final Design & Justification

Architecture of Information

- Find (search jobs) > Get ready (check) → Get started (apply).
- Persistent actions (Save, Review, Apply) remain in the same spot to lessen searching.

Progress Tab as the Keystone

- Browsing mode: Job History offers lightweight auditability and avoids duplication.
- Automation mode: The "is it stuck?" question is removed since the live stepper narrates work in real time.

Predictability Before Acting

Misclicks and anxiety are decreased by hovering over the Review ("edit before submitting") and Apply ("run automation and submit when valid") suggestions.

Engineering Decisions That Benefit User Experience

- JWT concerning cookies: eliminates flakiness in auth; perfect for MV3.
- The content script's state machine: Debugging is made easier by UI states mirroring code states.
- Selector registry: a single location for site updates.
- Loop guard and storage: chrome.storage.local maintains counter consistency; onModalClose() stops double listeners.

Effectiveness of the Solution

Problem & Objective

Online applications are simple to lose track of, repetitive, and opaque. The brief required a one-click experience: identify the form, fill it out, display a preview, submit, and maintain a record.

Scope This Term

Since every website behaves differently, we concentrated on mastering two platforms:

- Easy apply on LinkedIn
- Easy apply on Seek

We also looked at a subset of external websites.

User-Facing Design (What changed)

Progress tab (dual purpose):

- Browsing is your job history, complete with badges and totals, to prevent accidentally reapplying.
- Reviewing and applying: becomes a live stepper: Finding → Parsing → Pre-fill → Upload → Verifying → Sending → Result.

Before you click, tooltips describe what will happen when you hover over the Review/Apply indications (review = you can modify first; apply means automation runs and submits when valid). This decreased hesitancy and accidental clicks.

Effectiveness (Evidence & Metrics)

- Speed: average application time decreased from around 60 to 10 to 15 seconds (about 75–83% quicker).

- Reliability: With the move to JWT, the extension is responsible for zero auth/400 failures; client sites are the source of the remaining problems.
- Finalisation: Unlike Sprint 1, all supported runs reach the Submit/Outcome state.

Engineering Practices

Overview

This term, we wanted to develop it to be safe to extend, modular, and testable—not simply "make it work." In addition to specific external "normal apply" flows, we concentrated on Seek and LinkedIn Easy Apply and invested in procedures that would make the expansion dependable now and more straightforward to expand in the future.

Architecture & Modularity (MV3-friendly)

What we did:

- Distinct division of responsibilities. While site-specific functionality resides in adapters, shared capabilities (authentication, progress/history, storage, and state transitions) remain in the centre. This implies that LinkedIn and Seek are solely aware of their peculiarities and selections.
- Explicit state machine. Detect → Parse → Pre-fill → Upload → Validate → Submit → Outcome are the identified phases that the automation pipeline follows, and the user interface reflects these statuses. Behaviour is predictable and debuggable due to the close connection between the code and the user interface.

- Chrome MV3 compatibility. We don't rely on obsolete APIs since background/service logic, content scripts, and dynamic module loading adhere to Manifest V3 principles.

- Why it's sound engineering: MV3 alignment prevents future rework, the state machine eliminates uncertainty ("where did it fail?"), And modular code decreases the blast radius.

Code Quality & Maintainability

What we did:

- Selector registries plus adapters. A site's DOM hooks are all specified in one location. Instead of chasing string literals throughout the repository, we fix a single file when a job board modifies its HTML.

- Patterns of defence. To prevent endless loops or multiple submissions, we cap step retries and guard event listeners (attach once, delete on modal closing).

- Logs are readable and consistent in style. The code is easier to comprehend, and the runs are simpler to reconstruct using a single formatting/linting technique and human-oriented logs ("Looking for next button…," "Validating answers…").

- Less drift and quicker onboarding for newcomers to the codebase are why it's innovative engineering.

Security & Privacy by Design

What we did:

- JWT over cookies. To accommodate MV3 enhancements, we shifted to stateless tokens. This simplified client code and eliminated the sporadic 400/auth problems from early releases.

- Access with the least privilege. Only the bare minimum of session metadata is stored, and host permissions are scoped to where we automate.
- Sincere outcome recording. We record a job_apply_outcome only when a real terminal event (e.g., a specified error or a successful submit) occurs. Logs don't include sensitive responses.

- Impact: Since switching to JWT, the extension has not produced any auth/400 problems; instead, the client site is responsible for any remaining issues (such as outages or page modifications).

Reliability, Testing & Verification

What we did:

- Deterministic fluxes. Debugging is accelerated since each problem may be linked to a specified step (for example, "fails in VALIDATE on step 2") as the user interface mirrors the state machine.

- Coordinated manual E2E inspections. For Seek/LinkedIn, we kept a simple checklist that included uploads, validation failures, modal closure behaviour, explore → review → apply, and more.

- Contract checks for adapters. Each adapter checks its important selectors upon load, and if the DOM doesn't meet expectations, it fails quickly with a human-readable hint.

- Telemetry-lite. We have enough breadcrumbs from step-level console events (no PII) to replicate failures in the real world.

- Impact: 100% of flows achieve a Submit/Outcome state (Sprint 1 did not); end-to-end runs are now consistent.

Developer Experience & Documentation

What we did:

- Truly effective installation. Assessor friction is decreased with a brief, task-based install guidance (load unpacked extension → create JWT → test on Seek/LinkedIn).

- Small PRs and hygiene issues. To speed up reviews and reduce regressions, we limited the scope of the modifications (e.g., "Add live step indicator," "Introduce JWT," and "Guard modal listeners").

- Control scope as a discipline. On two platforms, we purposefully gave depth precedence over fragile breadth. This was not a shortcut, but a deliberate engineering trade-off.

As a result, it is simpler to execute, evaluate, and analyse.

UX Engineering (Safety, Clarity, Accessibility)

What we did:

- The system status is visible on the progress tab. There is no "black box" automation; users may see their location in real time.

- On Review/Apply, hover over the indicators. Before you click ("Review lets you edit first" vs. "Apply will run automation and submit when valid"), tooltips and subtle highlights inform you of what will happen.

- Preventing errors. Review provides a secure staging space for websites without native previews; Apply disables with a clear explanation if necessary information is absent.

- Where native previews are available. To reduce uncertainty and provide a consistent experience, we used the built-in previews on Seek and LinkedIn this term (as well as our Review process elsewhere).

Impact: less anxiety, fewer misclicks, and a more distinct mental image of the automation.

(1) Installation Manual

Our project was excluded from having to be dockerised and therefore the installation process is slightly different to most other projects. To install the jobGen.AI chrome extension currently we must first go to the github page and clone the current main branch into our code editor of choice. Once this step is finished, open chrome and under the puzzle piece in the top right of the tab, click manage extensions. Then, once on the extension manager page, simply click ‘load unpacked’ to reveal the file explorer and select the src directory of the extension to upload to the chrome extension manager.

Once complete, the chrome extension should look like this (shown below)

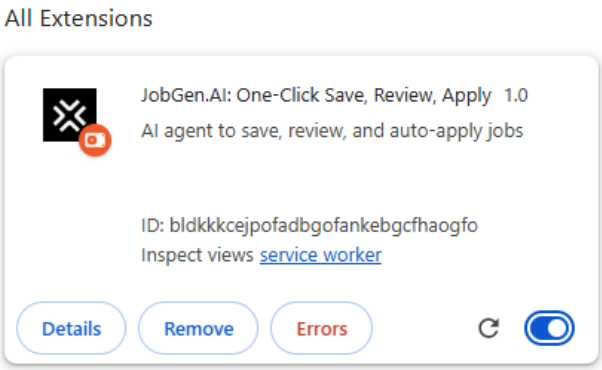


Figure 1: Snippet of chrome extension in the extension manager.

Now, all that is left to do is go to https://www.jobgen.ai/version-test/ and click ‘sign in’ at the top right to get a link for sign up to your email. From here, fill all the required pages to set up your profile and you should be set.

(2) System Architecture Diagram

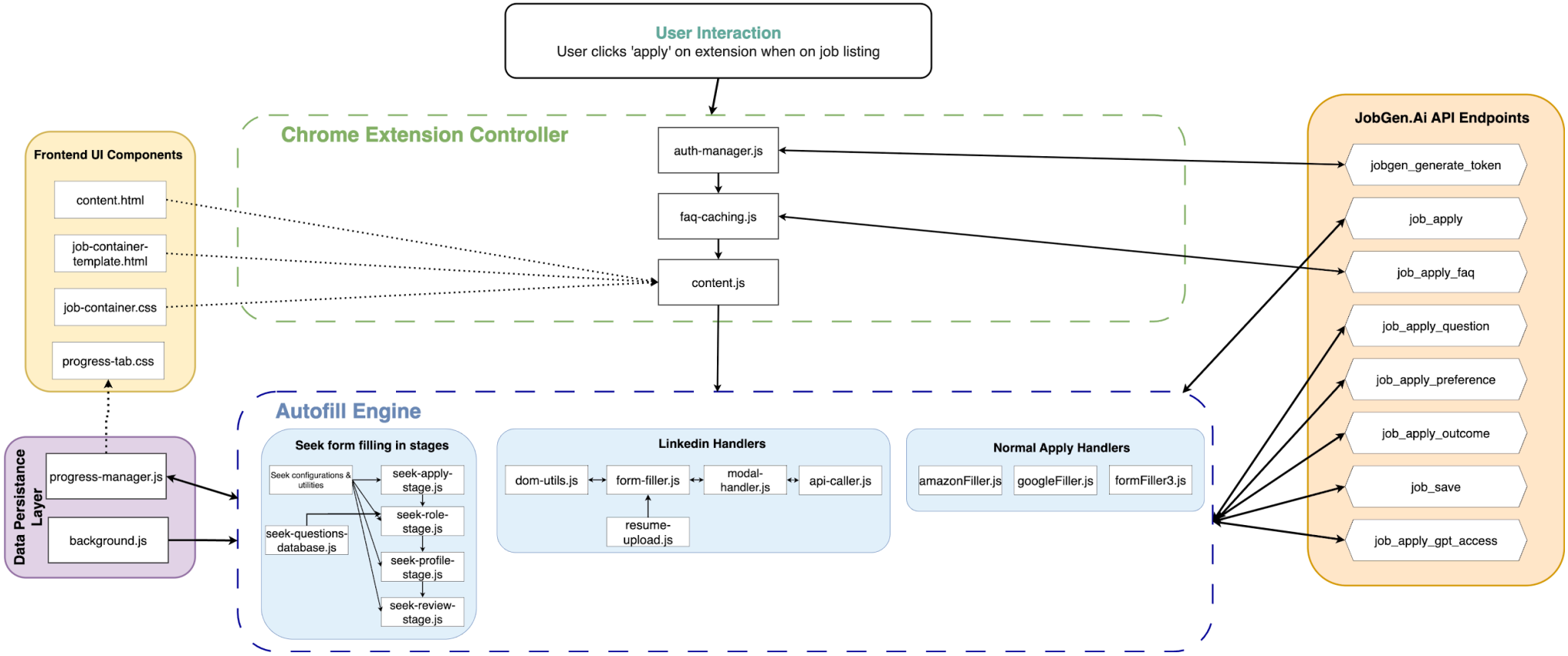


Figure 2

[Link for clearer diagram]

(3) Design Justifications

3.1 Initial Design Vision vs Final Implementation

Our initial design concept, based on the client's scope of work was the following:

1) The extension will detect application forms on the following sites
    a. LinkedIn
    b. Seek
    c. Indeed
    d. Naukri
    e. Glassdoor
2) And in each of these sites, handle the following questions:
    a. Personal Info: Name, email, Phone, Addresses
    b. Education
    c. Experience questions
    d. Expected Salary
    e. Work Rights
3) Furthermore, in these sites, we should automatically upload resumes without the user having to click any buttons.

While our final implementation includes many of the above requirements, it also is very different from what we initially planned for ourselves. Here is exactly what changed, and why:

Platform Evolution

Initial Design

Functional in 5 different job application sites, with tailored methods of auto filling in each site.

Problem Identified

After a few weeks of working on the extension, we realized that each site was way more work than initially thought and after weeks of working on LinkedIn, it was clear that it was impossible to implement this functionality in all 5 websites.

Solution Implemented

After combing through each of the sites, and how their job application process works, and after many client meetings, we agreed to only focus on LinkedIn and Seek. This was way more effective as it gives us a realistic workload for the timeframe that we have. If we were to do all 5 sites, all of them would be half-done and with limited functionality.

Code Evolution

Initial Design

As we were working on an already established codebase, we had to find the most effective way to add features to it that was easily maintainable. At first, 95% of the codes functionality was in a single file: content.js.

Problem Identified

As time went on, this file was growing and soon was over 2000 lines. This made the code extremely difficult to maintain and its readability was terrible. We were having issues with testing, bug fixing, and merging our code together was creating massive conflicts that took ages to fix.

Solution Implemented

We separated concerns into specialised modules. We tried our best to separate each new feature into a new file or a collection of files. This was extremely effective in simplifying the process of adding new features and made merging much less of a headache as there were less conflicts.

3.2 User Experience Evolution

Progress Tracking

Initial Design

Initially, the plan was to just handle the applications, the auto filling, and the resume upload for both sites which already took up a lot of our time. But our client highlighted a potential problem with our implementation.

Problem Identified

Since by design, this extension aims to speed up the process of applying to jobs. This could result in the user losing track of all the jobs they have applied to.

Solution Implemented

Created a ProgressManager class that uses local storage in chrome to keep track of all reviewed, saved, and applied jobs. This is all displayed in a new tab in the extension called ‘Progress’. There is also some basic job statistics and history displayed that provides a high level overview of the users progress.

Real-Time Feedback During Application

Initial Design

The extension was just static, with no ‘feedback’ given to the user when they perform an action. So for example if they were to click review in Linkedin, and for some reason the extension was working slow that day, there would be no visual indication that the extension was running.

Problem Identified

This could be problematic as users could think that the extension is frozen or broken when it isn’t, and they could start spamming buttons which could actually break some processes.

Solution Implemented

We added progress-tab.js for real time status updates during the application using custom events in javascript. These updates were designed to be visually appealing and easy to understand so users know exactly what stage the application is at now. This was very effective as it significantly improved user confidence and reduced abandonment rates.

3.3 Complex Algorithms and Tools

Dynamic Form Fields

Research and Planning

In order to have a dynamic autofilling feature, we had to analyse many job posts across linkedin and seek, and account for slight changes in language for the same answer. For example, a company could ask Do you require a sponsorship for a visa? Or Do you require financial assistance for a visa? This required us to do some serious planning in order to tackle these sort of questions appropriately, and without simply calling for the AI to answer everything.

Implementation Strategy

The most effective way we found to solve this issue was to check for specific keywords in the label of the question and if that returns true, then we return the answer for that question based on our Job_apply_faq API, which is essentially an object that contains all significant information of the user that they fill out during registration.
Here is an example of a check that we do to see if a field is asking for a mobile number. It contains keywords that is compared to the question, and ensures that the question isn’t actually asking for your phone country code.

Automatic Resume Uploads

Research and planning

One of the harder issues we came across was an automated resume upload system. This was because of chrome's own security policies that meant that extensions do not have access to a users locally stored files. This was a problem as when the 'upload resume' button is clicked, in most cases it opens up a file picker which is separate to chrome, and our extension cannot directly communicate with that to pick a resume. After lots of research, we discovered that we can sort of 'inject' a resume link directly into the html of the website, without ever having to click that upload resume button.

Implementation Strategy

We were given the job_apply API which takes in a job title, description, and other details about the job which is scraped from the job site. This is sent off as a request to the API which returns a resume link. In resume-upload.js, we call uploadFileFromURL. This downloads the file from the returned link as a blob and wraps it in a file object. The function then simulates a file upload by creating a DataTransfer object and adds the file to it. It locates the file input element in the site, and directly sets the file property to the new file. This method required a ton of research and trial and error.

---

(4) User-Driven Evaluation of Solution

Our product was initially described by the client as automating the job application process on job boards Seek, LinkedIn and Indeed through form detection and autofill. However, half-way through the project, our client reduced the scope to only LinkedIn and Seek which meant we could put more resources into the quality of the solutions rather than quantity.

4.1 - Functional requirements

In the end, we finalised easy apply for LinkedIn and quick apply for Seek, even adding an application progress tab to the extension (pictured below) which allows users to view the progress and outcome of jobGen.AI after applying to or reviewing a job application.

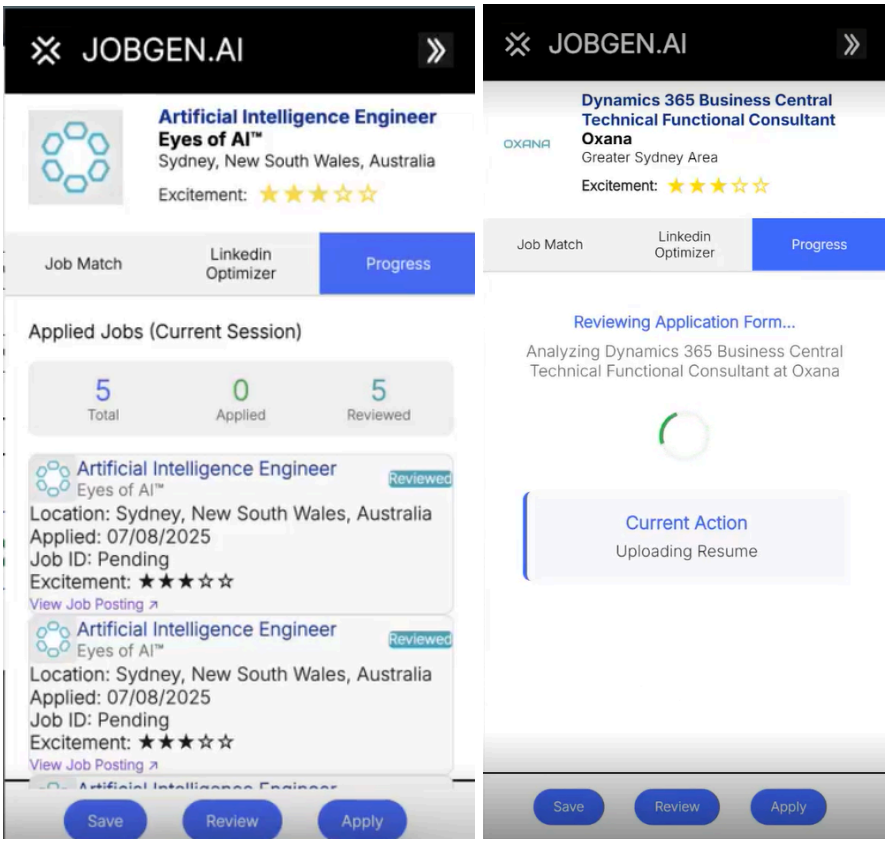


Figure 3: Progress tab of job extension.

On top of this we managed to implement the extension on a wide range of non-easy-apply applications in LinkedIn. This includes job applications where signing up was necessary to submit the job application. Below, the process from signing up to submitting an application is shown.

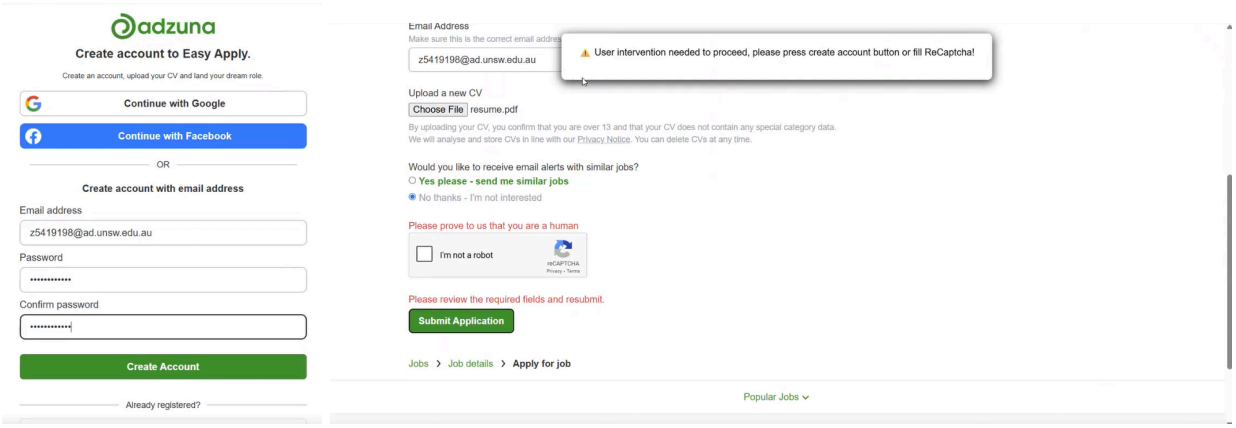


Figure 4: Applications to websites with signup pages.

To generate the user password, a random string generator was used for security purposes and this email would then be sent to the user's email through the email-js service for them to reset it manually on the job website as seen below.

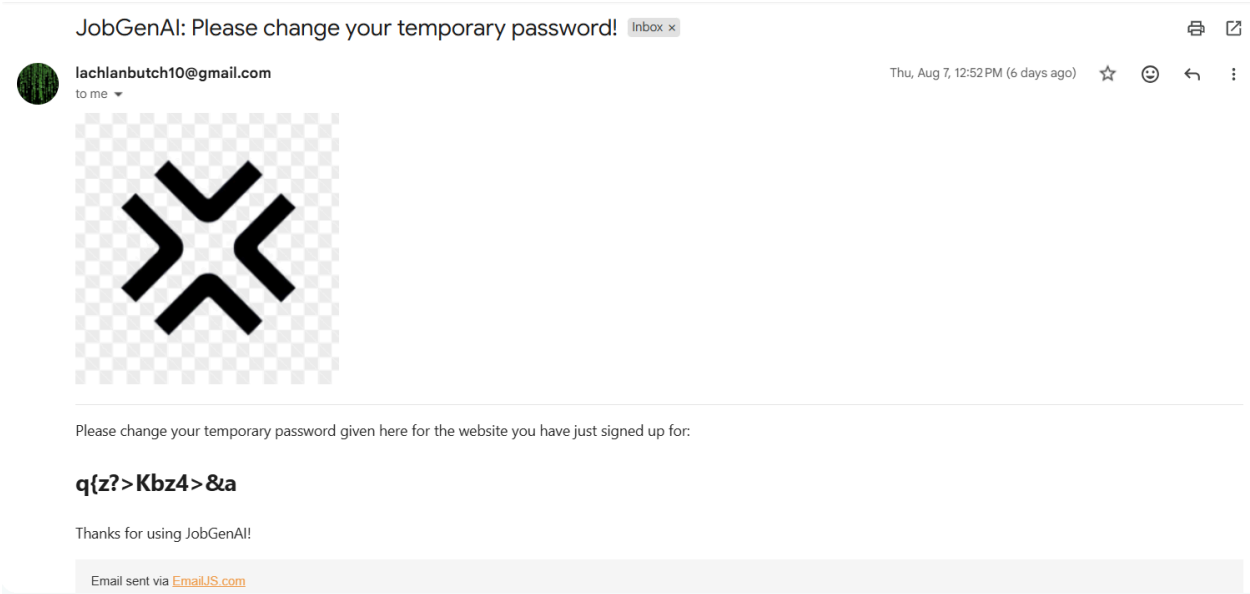
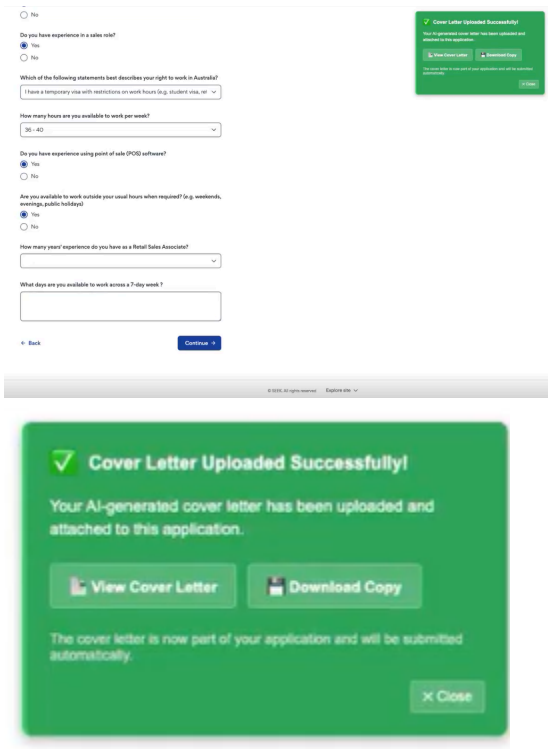


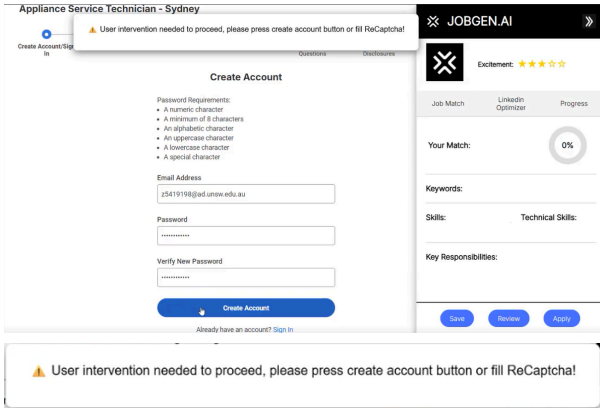
Figure 5: JobGenAI temporary password email.

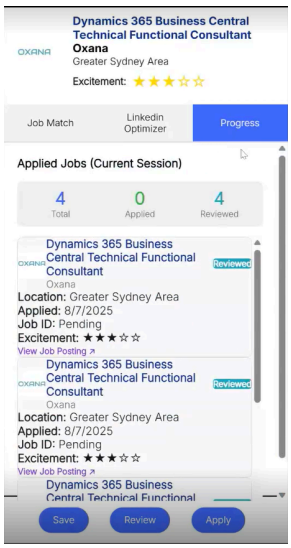
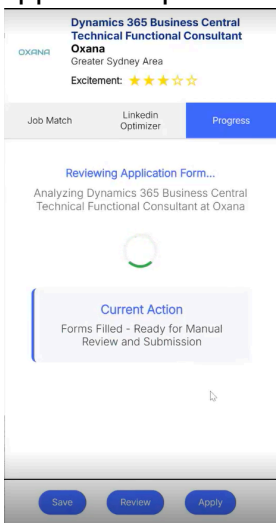
4.2 - Product specifications

This sections details the set of criteria or specifications which were set for us to complete from the client's scope and our effectiveness in meeting those specifications.

Specification	**Execution effectiveness**
Seek has 10 CV upload limit accounted for	This feature was implemented quite well as shown in section 4.1 for seek quick apply. However, regular applications in Seek have not been accounted for as we were not able to complete this feature in the allotted time frame.
LinkedIn easy apply automated	This requirement was fulfilled effectively by our extension which meets all checkboxes when it comes to each subtask listed below: - Detect form and input fields on the application pages. - Upload AI generated resume through Amazon Web Services (AWS) S3 bucket for improved security. - Fetch information from the API about the user and use this to fill form fields. - Submit application automatically and record submission
Seek quick apply automated	Similarly to LinkedIn easy apply, seek quick apply was implemented perfectly with our design boasting an impressive UI which captures users as seen below.

	 Our implementation here goes above and beyond the process of simply autofilling input and form fields by also giving us the ability to view and download the uploaded cover letter, something which Seek does not allow users to do.
LinkedIn non-easy-apply without sign in automated for most job listings	We managed to automate linkedIn non-easy-apply for the majority of websites. The exception to this were websites where form fields could not be identified on any page once the apply button was clicked.
Seek non-quick-apply automated for most job listings	Unfortunately due to time constraints we were unable to complete the seek non-quick-apply functionality. However, the ground work has already been layed out for this through the linkedIn non-easy-apply workflow which would follow a similar structure in terms of DOM traversal.
Sign-up functionality implemented for most websites	The sign up functionality proved to be quite difficult as many external job sites,

	including WorkDay, did not allow the autonomous creation of accounts, for obvious reasons. To get around this spam-prevention mechanism we simply added a pop up which prompts users to click any manual inputs which may interfere with the sign up process. This is shown below.  The password was randomly generated but due to the complexity of interfacing email-js with the DOM, we were not able to link the password in the job application field with the actual password that would be sent by email via email-js. This would be one major flaw in the system.
Extension has job history tab so users can track which jobs have been applied to	The job history tab was added to the scope later on and is implemented perfectly, showcasing brilliant UI and easy to navigate features as shown below.

	
Extension has job application progress tab so users can see which stage the application is in when being autofilled	This specification, similarly to the job application history tab, showcases a smart use of UI and logical flow. This tab simply shows the step in which the application is in for LinkedIn easy apply. However, this feature has not been implemented on Seek or LinkedIn normal apply and these would be the next steps to making this extension stand out even more. Below is a snapshot of the job application process tab. 
Extension is only active on job application	The execution of this feature was

websites	completed however more refinement is needed to ensure that the extension is active on all job listing websites. Currently, the extension only filters for keywords in the job application URL but in future we could look at keyword detection on the page and identify certain fields such as email or resume upload to confirm a job application page.

---

(5) Limitations and Future Work

During the development of our project, we encountered several technical, functional, and external limitations that influenced our design decisions and provided a clear direction for future improvements.

5.1 - Technical and Software Limitations

A significant limitation was encountered during the automation of web forms. We have identified two primary technical hurdles:

- **Dynamic elements**: We've had issues with detecting fields on websites similar to Amazon and Google, where the ids and classes of the fields are either dynamically generated or custom-made. Most of the time, the strings of field ids/names would be randomly generated e.g. 'name="B6fBgW_Ona"'. This made it difficult to detect fields for every application form.

- **Bot detection and CAPTCHAs**: We had encountered an issue with automating the resume upload for Google. Google has significant resources dedicated to prevent bot activities such as CAPTCHAs which is specifically designed to distinguish between human users and bots.

5.2 - Functional Limitations

Certain functionalities were deemed out of scope for the project or presented some technical barriers. We have documented these issues to inform the client and future developers:

- Account confirmation emails
- SMS confirmations
- Two-factor authentication (2FA)

Although these were not in the scope of work, we have attempted to automate the process of handling and verifying account confirmation emails however, this was

deemed impossible since we need user intervention to confirm their account. This is the case for SMS and 2FA as well.

5.3 - External Limitations and Backend Issues

The project's functionality is reliant on the stability of the backend. We have discovered a bug where the address field would occasionally be empty, which prevented us from updating our personal information and required us to refresh the page. This specific issue has been documented and will be discussed with the client for a resolution.

5.4 - Documentation and Knowledge Transfer

To address these limitations and support future development, we have created a few documents for future developers in the repository. The SETUP.md and README.md files provide a rundown of the project's core functionality, details of known limitations, and a clear guidance on the required packages and setup process. We have organised a couple meetings to explain the process to the client to ensure a seamless handover for future iterations.